MCT-454L Mobile Robotics

Lab 8: Building and Visualizing a Custom Mobile Robot using URDF

1. Objective

The objective of this lab was to design and construct a 3D robot model using the Unified Robot Description Format (URDF). The focus was on understanding the hierarchical relationship between links and joints, and visualizing the resulting transform (TF) tree within the ROS 2 RViz environment.

2. Robot Design & Customizations

Instead of a standard box-shaped robot, I developed a custom design optimized for internal component visualization and sensor mounting.

2.1 Mechanical Structure (Links)

- **Transparent Chassis (base_link):** The primary chassis was designed as a box with dimensions $0.4 \times 0.3 \times 0.1$ meters. A critical customization was the use of a semi-transparent material (Alpha < 1.0), allowing for the visualization of internal coordinate frames and the center of mass.
- **Differential Drive Wheels:** Two primary wheels were attached to the sides, modeled as cylinders.
- **Stability System:** A caster wheel was added to the front to provide a stable tripod base for the differential drive configuration.
- **Sensor Pillars:** Four vertical cylindrical pillars were added to the top surface of the chassis. These represent mounting points for future LiDAR and camera sensors, moving the design beyond a simple "box-on-wheels" configuration.

2.2 Joint Configuration

- **Continuous Joints:** The left and right wheels were defined using the continuous joint type, enabling 360-degree rotation along the Y-axis for mobility.
- **Fixed Joints:** The sensor pillars and the caster wheel were connected via fixed joints to ensure structural rigidity during movement.

3. Design Challenges & Problem Solving

3.1 The Omnidirectional Drive Ambition

Initially, we explored the possibility of an **Omnidirectional (Holonomic) Drive** system. However, several constraints were identified:

- **Mathematical Complexity:** True omni-wheels (Mecanum) require complex URDF definitions and specific Gazebo plugins for friction modeling that were outside the scope of this introductory lab.
- **Stack Compatibility:** To maintain compatibility with the Nav2 stack and AMCL localization used in Lab 7, we reverted to a **Differential Drive** baseline. This ensures the robot can utilize the standard motion controllers without requiring custom kinematics.

3.2 Directory and Launch Errors

A significant hurdle was encountered when launching the visualization. The system initially failed to find the URDF because:

1. **Relative Pathing:** The display.launch.py script defaults to system directories. This was resolved by using **Absolute Paths** (/home/rayan/Rayan_ros2_ws/...) to point directly to the local workspace.
2. **Typographical Errors:** A directory in the package was accidentally named lanch instead of launch. This was manually corrected to comply with ROS 2 naming conventions, ensuring the build system could correctly index the package files.

3.3 Manual Errors

The provided lab manual contained a syntax error in the sample URDF code, where a camera joint incorrectly referenced a left_wheel as its child link. I manually audited the XML structure to ensure every joint correctly mapped to its corresponding physical link.

4. Technical Visualizations

- **TF Tree Generation:** Using ros2 run tf2_tools view_frames, I generated a hierarchical tree showing the relationship from base_link -> wheel_joints -> wheels. This confirmed that the transformations are correctly propagated through the robot's skeleton.
- **Joint State Interaction:** Using the joint_state_publisher_gui, I verified that moving the sliders correctly rotated the wheels in the RViz environment without detaching them from the chassis.

5. Conclusion

Lab 8 provided a fundamental understanding of how ROS 2 represents physical hardware. By building a custom model rather than using a stock TurtleBot, I learned how to manage coordinate frames (TFs) and material properties. The experience highlighted that while URDF is an XML format, the underlying logic requires strict adherence to mechanical hierarchies and precise file-system management. This model serves as the physical "body" that will be integrated with the "brains" developed in the previous navigation labs.